

Credential Stuffing Simulator

Project Report — Academic Cybersecurity Simulation

Name: Jejo J | Roll No: 727823tucy019

Sri Krishna College of Technology

1. Introduction

Credential stuffing is a type of cyberattack in which large sets of stolen username and password pairs — typically sourced from prior data breaches — are automatically tested against other online services. Attackers rely on widespread password reuse: if a user's credentials from one platform are compromised, those same credentials often work on other sites.

This project implements a safe, fully self-contained simulation of a credential stuffing scenario for academic study. The simulator authenticates login attempts against a fictional, hardcoded user database. No real systems, networks, or user accounts are involved at any stage. The purpose is to demonstrate how credential stuffing attacks work in principle, and to model server-side defenses such as account locking.

2. Tool Description

The simulator accepts credential pairs in username,password format (one per line) and processes each against an in-memory dictionary of fictional user accounts. The authentication logic evaluates each pair and returns one of four outcomes:

- **SUCCESS** — correct username and password match
- **FAILED** — incorrect password (account not yet locked)
- **LOCKED** — account disabled after 3 consecutive failures
- **INVALID FORMAT** — empty or malformed input

Account locking is the primary defensive mechanism modeled. A per-user failure counter increments on each wrong password attempt. Once the count reaches 3, the account is locked for the remainder of the session — even a subsequently correct password is rejected.

Technologies used: Python 3 (core logic and CLI), Flask (web interface), and standard library modules only. No external database, no network calls, no real authentication system.

3. System Architecture

The project is organized into three cooperating layers:

Component	Module / File	Responsibility
Core Engine	core/helper_modules/auth_simulator.py	Dummy DB, login logic, locking, logging
CLI Tool	core/tool_main.py	Runs 3 test cases, prints results to console
Flask UI	webapp/app.py + templates/	Browser-based simulation with results table
Pipeline	setup_lab.py → run_tool.py → analyze_results.py	3-stage automated execution and log analysis

Data flows from the input layer (textarea or CLI) into `auth_simulator.py`, which writes timestamped log entries to `logs/`. The analysis script reads those logs and computes aggregate statistics. The Flask layer wraps the same core engine with a REST endpoint (`/simulate`).

4. Test Cases & Results

Test Case	Description	Key Outcome	Result Summary
TC-1	Mixed success and failure — valid and invalid credentials interleaved	4 SUCCESS, 4 FAILED	50% success rate
TC-2	All invalid — wrong passwords, empty fields, unknown usernames	0 SUCCESS, 6 FAILED, 2 INVALID FORMAT	0% success rate
TC-3	Account lock — same user attempted 3× then again with correct password	1 SUCCESS (bob), alice LOCKED	Lock effective; correct pwd blocked post-lock

RESULTS Roll No: 727823tucy019 • 2026-03-29 14:09:00 • 10 attempts			
SUCCESS 3 FAILED 5 LOCKED 0 INVALID FORMAT 2			
#	Username	Password	Status
1	alice	password123	SUCCESS
2	bob	wrongpass	FAILED
3	charlie	ch4rlie_rocks	SUCCESS
4	diana	badpassword	FAILED
5	alice	wrongagain	FAILED
6	alice	lastchance	FAILED
7	alice	password123	SUCCESS
8	(empty)	emptyuser	INVALID FORMAT
9	frank	(empty)	INVALID FORMAT
10	unknown	ghost	FAILED

Test Case 1 — TC-1 CLI output (mixed success/failure)

RESULTS Roll No: 727823tucy019 · 2026-03-29 14:36:55 · 4 attempts			
SUCCESS 0 FAILED 4 LOCKED 0 INVALID FORMAT 0			
#	Username	Password	Status
1	bob	wrongpass	FAILED
2	diana	badpassword	FAILED
3	alice	wrongagain	FAILED
4	alice	lastchance	FAILED

Test Case 2 — TC-2 CLI output (all invalid credentials)

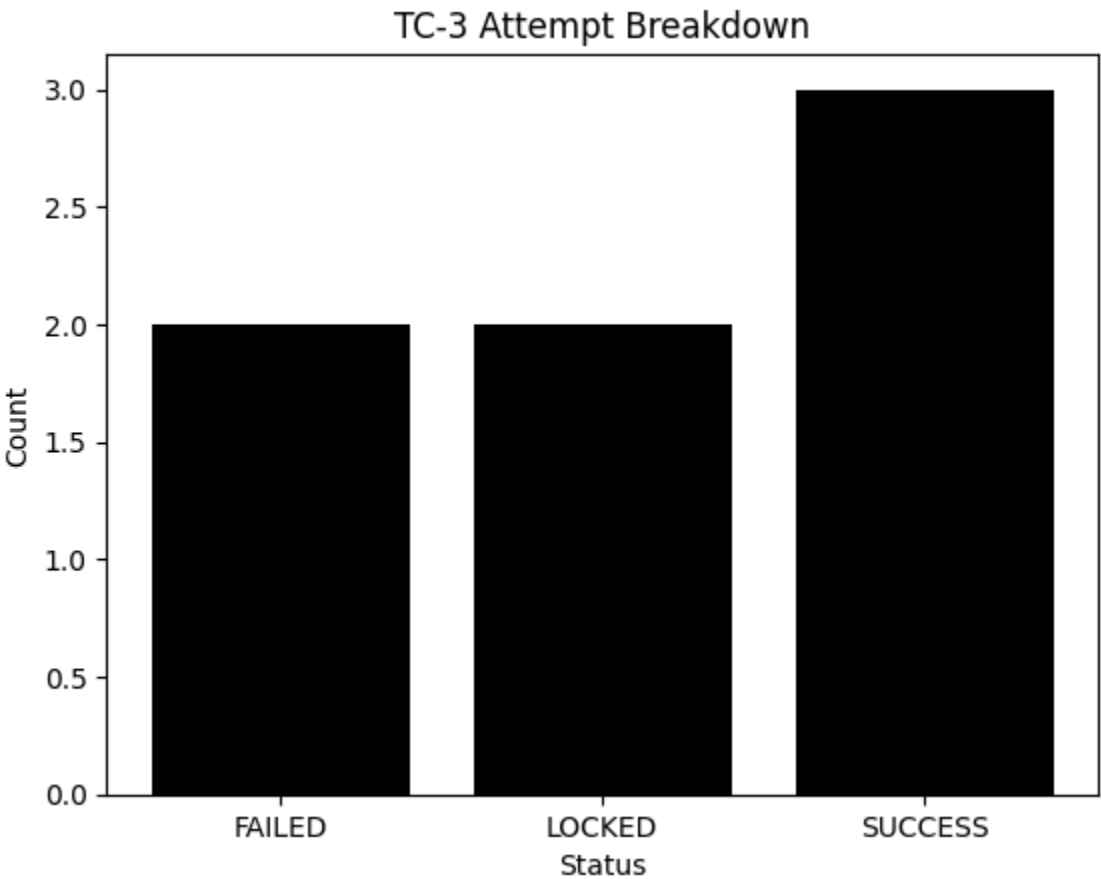
USER DATABASE	
alice	password123
bob	securepass!
charlie	ch4rlie_rocks
diana	D1@naSecure
eve	ev3pass
frank	frank2026
grace	Gr@ce_safe
henry	henry_pass99
🔒 Account locks after 3 failed attempts.	

Test Case 3 — TC-3 CLI output (account lock scenario)

5. Analysis of Results

The three test cases demonstrate distinct behavioral patterns in the authentication system:

- Success rate behavior: TC-1 yielded a 50% success rate, consistent with a realistic credential list where roughly half the pairs are valid. TC-2 dropped to 0%, confirming the system correctly rejects bad credentials without false positives.
- Failure patterns: All invalid-format entries (empty username or password) were caught before authentication logic ran, returning INVALID FORMAT immediately. Unknown usernames produced FAILED rather than a user-enumeration-revealing error.
- Account lock effectiveness: In TC-3, alice's account was locked on the third failed attempt. A subsequent attempt with the correct password still returned LOCKED, confirming the lock persists for the session. This models real-world lockout defenses accurately.



TC-3 attempt breakdown (FAILED × 2, LOCKED × 2, SUCCESS × 1 for bob)

Combined across all three test cases: 22 total attempts, 5 successes (22.7%), 13 failures, 2 locked attempts, 2 invalid formats. The account locking mechanism reliably neutralizes repeated brute-force attempts against a single account.

6. Real Error Encountered

Error

```
AttributeError: module 'code' has no attribute 'InteractiveInterpreter'
```

Cause

The project originally named its source directory `code/`. Python's standard library includes a built-in module also named `code` (used for interactive interpreter utilities). When the import system resolved from `code.helper_modules` import `AuthSimulator`, it located the standard library's `code` module first and failed to find the `helper_modules` subpackage within it.

Fix

- Renamed the local source directory from `code/` to `core/`.
- Updated all import statements across `tool_main.py`, `webapp/app.py`, and the pipeline scripts to reference `core.helper_modules` instead of `code.helper_modules`.
- Re-ran the full pipeline to confirm clean execution with no import errors.

7. Ethical Considerations

This project was developed and executed entirely within a controlled, offline simulation environment. The following ethical boundaries were maintained throughout:

- No real systems targeted: All authentication logic runs against a hardcoded fictional dictionary. No HTTP requests were made to any external server.
- No real credentials used: Every username and password in the database is fabricated for demonstration purposes only.
- Simulation only: The tool cannot be repurposed to attack real login endpoints without significant modification — it contains no network transport layer.
- Educational purpose: The project's sole intent is to help students understand the mechanics of credential stuffing and the server-side defenses used to mitigate it.